

ORM SQLAlchemy

Основы работы с базами данных в Python: от PEP 249 до выбора ORM

Зачем это нужно

SQLAlchemy — библиотека для Python, которая позволяет работать с базой данных через объекты и классы, а не через ручное написание SQL (Structured Query Language — язык структурированных запросов). Она заметно упрощает разработку и поддержку приложений, которые используют базы данных.

В теме разберём: стандарт PEP 249, на котором строится работа с базами в Python, проблемы прямой работы с SQL, зачем нужны ORM (Object-Relational Mapping — объектно-реляционное отображение), обзор популярных ORM-библиотек и первые шаги в SQLAlchemy.

Стандарт PEP 249

PEP 249 (Python Enhancement Proposal — предложение по улучшению Python) 249, он же Python Database API Specification v2.0 — это стандарт, который описывает единый интерфейс для работы с разными базами данных из Python. Благодаря ему модули для разных СУБД (систем управления базами данных) выглядят и работают похожим образом.

Стандарт определяет два ключевых объекта: **Connection** (соединение с базой) и **Cursor** (курсор для выполнения запросов).

Объект Connection

Метод	Что делает
<code>close()</code>	Закрывает соединение с базой, освобождает ресурсы
<code>commit()</code>	Подтверждает (применяет) все изменения текущей транзакции
<code>rollback()</code>	Откатывает изменения текущей транзакции (не все базы это поддерживают)
<code>cursor()</code>	Создаёт новый объект Cursor для выполнения запросов

Объект Cursor

Метод	Что делает
<code>execute(sql, params)</code>	Выполняет один SQL-запрос, параметры передаются отдельно
<code>executemany(sql, seq)</code>	Выполняет один и тот же запрос с разными наборами параметров
<code>fetchone()</code>	Возвращает следующую строку результата
<code>fetchmany(size)</code>	Возвращает несколько строк результата
<code>fetchall()</code>	Возвращает все строки результата
<code>close()</code>	Закрывает курсор

Пример по PEP 249 (SQLite)

Соединение удобно оформить как менеджер контекста, чтобы оно всегда корректно закрывалось:

```
import sqlite3
from contextlib import contextmanager

database = './test.db'

@contextmanager
def create_connection(db_file):
    conn = sqlite3.connect(db_file)
    yield conn
    conn.rollback()
    conn.close()
```

Дальше — типовой набор действий: создать таблицу, добавить запись, получить данные. Во всех случаях схема одна: открыть курсор → выполнить SQL → `commit()` для изменений → закрыть курсор.

```
def create_project(conn, project):
    sql = "INSERT INTO projects(name, begin_date, end_date) VALUES (?, ?, ?)"
    cur = conn.cursor()
    cur.execute(sql, project)
    conn.commit()
    return cur.lastrowid # id последней добавленной строки

def select_projects(conn):
    cur = conn.cursor()
    cur.execute("SELECT * FROM projects")
    return cur.fetchall()
```

Важно: у курсора после INSERT/UPDATE/DELETE доступен атрибут `lastrowid` — id последней строки, с которой он работал. Обновление и удаление данных строятся по такому же принципу: `UPDATE ... WHERE id = ?` и `DELETE ... WHERE id = ?`, выполненные через `execute()`.

Проблема прямой работы с SQL

Реляционные базы данных (RDB — Relational Database) — стандарт хранения данных, и почти все они «общаются» на SQL. Поэтому в приложении неизбежно появляется код на SQL вперемешку с кодом на Python. Это усложняет поддержку: логика частично на одном языке, частично на другом, код хуже читается.

Главная боль — изменения схемы. Если нужно добавить колонку или связь с новой таблицей, приходится переписывать SQL-запрос и все места кода, которые с ним работают. Это легко приводит к ошибкам и плохо масштабируется на сложных проектах.

ORM (Object-Relational Mapping) — техника программирования, которая позволяет работать с базой данных через объекты и классы вместо «чистого» SQL. Структура данных описывается один раз (как Python-класс), а сами SQL-запросы ORM генерирует автоматически.

Обзор ORM в Python

Библиотека	Особенности
SQLAlchemy	Самая популярная и гибкая. Два уровня: Core (близко к SQL) и ORM (объекты). Хорошо работает с Flask и FastAPI
Peewee	Лёгкая и простая, меньше возможностей. Хороша для небольших проектов и простого CRUD (Create, Read, Update, Delete — создание, чтение, обновление, удаление)
Tortoise ORM	Асинхронная, синтаксис похож на Django ORM. Отлично подходит для FastAPI
SQLModel	От создателя FastAPI. Сочетает SQLAlchemy и Pydantic (валидация данных), удобна для API-схем

Почему на курсе выбрали SQLAlchemy

- **Гибкость** — есть низкоуровневый Core (SQL-подобный API) и высокоуровневый ORM (работа через объекты), можно выбирать под задачу.
- **Широкая поддержка СУБД** — PostgreSQL, MySQL, SQLite, Oracle и другие.
- **Производительность** — есть механизмы оптимизации запросов и кеширования.
- **Зрелость и надёжность** — проверена временем, большое сообщество, много готовых решений при проблемах.

Для маленьких проектов с простым CRUD иногда удобнее Peewee, а для проектов, плотно завязанных на конкретный фреймворк — специализированные ORM вроде Django ORM. Но для большинства задач, особенно масштабируемых, SQLAlchemy — оптимальный баланс мощности и гибкости.

Установка и первое подключение

```
poetry add sqlalchemy
```

Работа с SQLAlchemy начинается с объекта **engine** — он инкапсулирует доступ к базе данных. Для его создания нужна строка подключения (connection string) с данными о базе.

```
from sqlalchemy import create_engine

engine = create_engine('sqlite:///memory:', echo=True)
```

В примере используется SQLite в памяти — данные не сохраняются на диск, это удобно для тестов и обучения. Параметр `echo=True` включает вывод генерируемых SQL-запросов в консоль — полезно, чтобы видеть, что реально делает ORM «под капотом».

Уровни SQLAlchemy: **Core** — низкий уровень, код очень близок к SQL, нужен только для специфичных задач. **ORM** — высокий уровень, основной способ работы: таблицы описываются как Python-классы (модели), а взаимодействие с данными идёт через сессии (session).

Работа на уровне Core

Core — способ работать с базой на уровне, близком к SQL, но через Python-объекты вместо строк с запросами. Чтобы связать структуру базы и её описание в коде, используется объект **MetaData** — реестр всех таблиц приложения.

```
from sqlalchemy import MetaData

metadata = MetaData()
```

Описание таблиц

Таблицы описываются через `Table`, столбцы — через `Column`. Пример: таблицы `users` и `addresses`, связанные внешним ключом.

```
from sqlalchemy import Table, Column, Integer, String, ForeignKey

users = Table('users', metadata,
    Column('id', Integer, primary_key=True),
    Column('name', String),
    Column('fullname', String),
)

addresses = Table('addresses', metadata,
    Column('id', Integer, primary_key=True),
    Column('user_id', Integer, ForeignKey('users.id')),
    Column('email_address', String, nullable=False)
)
```

Чтобы реально создать эти таблицы в базе, объекту `metadata` передают `engine`:

```
metadata.create_all(engine)
```

Вставка и выборка данных

`insert()` и `select()` только строят SQL-выражение — выполняется оно через соединение (`connection`):

```
from sqlalchemy.sql import select

with engine.connect() as conn:
    ins = users.insert().values(name='jack', fullname='Jack Jones')
    conn.execute(ins)

    result = conn.execute(select(users))
    for row in result:
        print(row)  # (1, 'jack', 'Jack Jones')
```

`echo=True` в `create_engine` выводит в консоль весь SQL, который реально выполняется «под капотом»: проверку существования таблиц, `CREATE TABLE`, `INSERT`, `SELECT` и подтверждение транзакции. Удобно для отладки и понимания, что именно генерирует ORM.

Core — мощный, но не самый удобный инструмент, нужен в основном для нетипичных задач. В повседневной разработке чаще используют более высокий уровень — **модели**.

Таблицы и модели (ORM)

В ORM-подходе таблица описывается как обычный Python-класс — **модель**. Атрибуты класса соответствуют столбцам таблицы.

Старый синтаксис (ещё встречается в чужом коде)

```

from sqlalchemy import Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

class User(Base):
    __tablename__ = 'users'

    id = Column(Integer, primary_key=True)
    name = Column(String(50))
    email = Column(String(120), unique=True)

```

Актуальный синтаксис (SQLAlchemy 2.0+)

С версии 2.0 рекомендуется использовать `Mapped` и `mapped_column()` — тип столбца при этом связан с типом Python:

```

from sqlalchemy import Integer, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = 'users'

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(50))
    email: Mapped[str] = mapped_column(String(120), unique=True)

```

Класс `Base` — основа для всех моделей. Именно он отвечает за синхронизацию Python-классов и таблиц базы данных. Все модели приложения наследуются от него.

Основные типы данных

Тип SQLAlchemy	Что хранит
<code>Integer</code>	Целое число
<code>String(length)</code>	Строка с указанной максимальной длиной
<code>Float</code>	Число с плавающей точкой
<code>Boolean</code>	Логическое значение (<code>True</code> / <code>False</code>)
<code>Date</code>	Дата
<code>DateTime</code>	Дата и время
<code>Enum</code>	Перечень допустимых значений

Связи между моделями (relationships)

Один-ко-многим

Например, у одного пользователя может быть много постов. Связь описывается функцией `relationship()` с двух сторон, а синхронизируются они параметром `back_populates`.

```

from sqlalchemy import ForeignKey, Integer, String
from sqlalchemy.orm import DeclarativeBase, Mapped, relationship, mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = 'users'

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(50))
    posts: Mapped[list['Post']] = relationship(back_populates='author')

class Post(Base):
    __tablename__ = 'posts'

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(100))
    user_id: Mapped[int] = mapped_column(ForeignKey('users.id'))
    author: Mapped['User'] = relationship(back_populates='posts')

```

Атрибут `posts` у `User` — список постов, атрибут `author` у `Post` — обратная ссылка на пользователя. Так можно строить запросы «в обе стороны».

Многие-ко-многим

Например, пользователь может состоять в нескольких группах, а группа содержит много пользователей. Такую связь напрямую не описать — нужна промежуточная таблица-association и параметр `secondary` в `relationship()`.

```

from sqlalchemy import Column, Integer, ForeignKey, Table, PrimaryKeyConstraint
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

# таблица-связка: только внешние ключи, без своей модели
association_table = Table(
    'association', Base.metadata,
    Column('user_id', Integer, ForeignKey('users.id', ondelete='CASCADE')),
    Column('group_id', Integer, ForeignKey('groups.id', ondelete='CASCADE')),
    PrimaryKeyConstraint('user_id', 'group_id'), # составной первичный ключ
)

class User(Base):
    __tablename__ = 'users'
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    groups: Mapped[list['Group']] = relationship(secondary=association_table, back_populates='users')

class Group(Base):
    __tablename__ = 'groups'
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    users: Mapped[list['User']] = relationship(secondary=association_table, back_populates='groups')

# использование:
user1.groups.extend([group1, group2])
session.add_all([user1, group1, group2])
session.commit()

```

`PrimaryKeyConstraint` задаёт составной первичный ключ из пары `user_id` + `group_id` — так не получится добавить дублирующую связь. Параметр `ondelete='CASCADE'` у внешних ключей удаляет строки связи

автоматически, если удалён пользователь или группа.

Один-к-одному

Строится на основе один-ко-многим, но с параметром `uselist=False` — тогда вместо списка будет один объект:

```
class User(Base):
    __tablename__ = 'users'
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    userinfo: Mapped['UserInfo'] = relationship('UserInfo', backref='user', uselist=False)

class UserInfo(Base):
    __tablename__ = 'userinfo'
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    phone: Mapped[str] = mapped_column(String(11))
    user_id: Mapped[int] = mapped_column(ForeignKey('users.id'))
```

`backref` — более старый аналог `back_populates`. Разница: `backref` достаточно указать в одной модели, и обратная связь появится автоматически во второй; `back_populates` нужно прописывать явно в обеих моделях.

Каскадные операции

Каскад определяет, что происходит со связанными записями при изменении или удалении основной. Есть два независимых уровня, которые не заменяют, а дополняют друг друга.

Уровень базы данных: `onDelete` / `onupdate`

Указываются прямо в `ForeignKey` и работают силами самой СУБД (даже если запрос пришёл не через SQLAlchemy):

```
user_id: Mapped[int] = mapped_column(
    ForeignKey('users.id', onDelete='CASCADE', onupdate='CASCADE')
)
```

`onDelete='CASCADE'` — при удалении пользователя автоматически удалятся его посты. `onupdate='CASCADE'` — при изменении первичного ключа пользователя обновится и внешний ключ в постах.

Уровень ORM: параметр `cascade` в `relationship()`

Управляет тем, что SQLAlchemy делает со связанными Python-объектами в рамках сессии:

Значение	Что делает
<code>save-update</code>	При сохранении/обновлении основного объекта автоматически сохраняются и связанные
<code>delete</code>	При удалении основного объекта удаляются и все связанные с ним
<code>delete-orphan</code>	Связанный объект удаляется, если он «отвязан» от основного (убран из списка/коллекции) и больше ни с чем не связан
<code>merge</code>	При объединении состояния основного объекта (<code>merge()</code>) объединяется и состояние связанных
<code>expunge</code>	При удалении основного объекта из сессии связанные объекты тоже убираются из сессии

```
posts: Mapped[list['Post']] = relationship('Post', back_populates='user', cascade='delete-orphan')
```

Разница простыми словами: `ondelete` / `onupdate` — это правило самой базы данных, работает всегда. `cascade` — это правило SQLAlchemy для работы с объектами в сессии. На практике часто используют оба, под разные задачи.

Механизм сессий (Session)

Сессия — это контекст, в котором SQLAlchemy отслеживает объекты и синхронизирует их с базой данных. Она отвечает за четыре вещи:

- **Жизненный цикл объектов** — сессия помнит, какие объекты добавлены, изменены или удалены
- **Транзакции** — все изменения применяются атомарно: либо все, либо ни одного (откат при ошибке)
- **Кэширование** — уже загруженные объекты не запрашиваются у базы повторно
- **Ленивая загрузка (lazy loading)** — связанные объекты подгружаются из базы только тогда, когда к ним реально обращаются

```
from sqlalchemy import create_engine
from sqlalchemy.orm import Session

engine = create_engine('sqlite:///example.db')
session = Session(engine)
```

Добавление, изменение, удаление

Изменения копятся в сессии и реально попадают в базу только после `commit()` :

```
# добавление
user = User(name='John Doe', email='john@example.com')
session.add(user)
session.commit()

# изменение
user = session.get(User, 1)
user.name = 'John Smith'
session.commit()

# удаление
user = session.get(User, 1)
session.delete(user)
session.commit()
```

`session.get(Модель, id)` — способ получить объект по первичному ключу. Важно: пока не вызван `commit()` , изменения существуют только внутри сессии и в базу не попали.

На практике сессию удобно открывать через менеджер контекста `with` — тогда она сама закроется:

```
with Session(engine) as session:
    user = User(name='John Doe', email='john@example.com')
    session.add(user)
    session.commit()
```

Запросы через сессию (query)

Базовый запрос строится через `session.query(Модель)` :

```
users = session.query(User).all() # все пользователи
```

Фильтрация, сортировка, пагинация

Метод	Что делает
<code>filter(User.age > 18)</code>	Фильтрация по условию (можно использовать сравнения, любые выражения)
<code>filter_by(name='John')</code>	Фильтрация по точному значению поля
<code>limit(10)</code>	Ограничивает результат первыми 10 строками
<code>offset(20)</code>	Пропускает первые 20 строк (вместе с <code>limit()</code> — пагинация)
<code>distinct()</code>	Убирает дубликаты из результата
<code>order_by(User.name.desc())</code>	Сортировка; по умолчанию по возрастанию, <code>.desc()</code> — по убыванию
<code>group_by(User.name)</code>	Группировка строк, обычно вместе с агрегатными функциями
<code>having(...)</code>	Фильтрация уже после группировки (например, по <code>func.count()</code>)

```
from sqlalchemy.sql import func

# пагинация: строки с 21-й по 30-ю
users = session.query(User).offset(20).limit(10).all()

# группировка + подсчёт одинаковых имён
result = (
    session.query(User.name, func.count(User.id))
    .group_by(User.name)
    .having(func.count(User.id) > 1)
    .all()
)
```

Как получить результат

Метод	Возвращает
<code>all()</code>	Список всех подходящих объектов
<code>first()</code>	Первый объект или <code>None</code> , если ничего не найдено
<code>one()</code>	Ровно один объект; ошибка, если найдено 0 или больше 1
<code>one_or_none()</code>	Один объект или <code>None</code> ; ошибка, если найдено больше 1
<code>scalar()</code>	Одно значение из первой строки (обычно с агрегатными функциями)

JOIN-запросы

Тип JOIN	Что возвращает
INNER JOIN	Только строки, для которых есть совпадение в обеих таблицах
LEFT OUTER JOIN	Все строки левой таблицы + совпадения справа (или <code>NULL</code> , если совпадений нет)
RIGHT OUTER JOIN	Все строки правой таблицы + совпадения слева
FULL OUTER JOIN	Все строки из обеих таблиц, совпавшие и нет

```
# INNER JOIN — SQLAlchemy сам найдёт связь по внешнему ключу
result = session.query(User, Address).join(Address)

# INNER JOIN с явным условием — полезно для нестандартных связей
result = session.query(User, Address).join(Address, User.id == Address.user_id)

# несколько JOIN подряд
result = session.query(User, Address, Order).join(Address).join(Order)

# LEFT OUTER JOIN — вернёт всех пользователей, даже без адреса
result = session.query(User, Address).outerjoin(Address)
```

RIGHT и FULL OUTER JOIN в SQLAlchemy отдельных методов не имеют: RIGHT получают, поменяв таблицы местами и используя `outerjoin(..., isouter=True)`; FULL — через `outerjoin(Address, full=True)`. Важно: не все базы данных поддерживают FULL OUTER JOIN (например, SQLite — нет), поэтому перед использованием стоит проверить возможности своей СУБД.

Если нужно явно задать, с какой таблицы начинается построение запроса (порядок присоединений), используют `select_from()`:

```
result = (
    session.query(User, Order, Address)
    .select_from(Order)
    .join(User)
    .join(Address)
)
```

Здесь запрос стартует от таблицы `Order`, а не от первой модели в `query()`. Это не влияет на то, какие колонки выбираются — только на порядок построения JOIN.

Метод `execute()` — запросы без сессий

В SQLAlchemy 2.0 появился альтернативный способ выполнять запросы — напрямую через `execute()`, без создания сессии. Это Core-стиль: код ближе к SQL, выполнение запроса происходит явно и не зависит от механизма сессии.

Это не замена сессиям, а дополнение. Сессия по-прежнему нужна, когда важно отслеживать состояние объектов и работать с ними как с Python-объектами. `execute()` удобен для простых запросов, где такое отслеживание не требуется.

```
from sqlalchemy import create_engine, text

engine = create_engine('sqlite:///example.db')

with engine.connect() as connection:
    result = connection.execute(text("SELECT * FROM users"))
    for row in result:
        print(row)
```

`text()` оборачивает «сырой» SQL в виде строки. Но для CRUD-операций удобнее использовать не строки, а конструкции `insert()`, `select()`, `update()`, `delete()` — тогда запрос строится через Python-объекты, как и в Core.

INSERT

```

from sqlalchemy import insert

with engine.connect() as connection:
    stmt = insert(User).values([
        {'name': 'John', 'age': 30},
        {'name': 'Jane', 'age': 25},
    ])
    connection.execute(stmt)
    connection.commit()

```

SELECT и фильтрация

```

from sqlalchemy import select

with engine.connect() as connection:
    stmt = select(User).where(User.age > 30)
    result = connection.execute(stmt)
    for user in result.scalars():
        print(user)

```

`result.scalars()` возвращает сами объекты модели (а не «сырые» строки-кортежи). У результата доступны те же методы, что и у сессионных запросов: `all()`, `first()`, `one()`, `one_or_none()`.

Условия в `where()` строятся операторами сравнения:

Оператор	Значение
<code>=</code> , <code>≠</code>	равно / не равно
<code><</code> , <code>></code> , <code>≤</code> , <code>≥</code>	сравнение чисел/дат
<code>in([...])</code>	значение входит в список
<code>like(...)</code> / <code>ilike(...)</code>	совпадение по шаблону (<code>ilike</code> — без учёта регистра)

```

from sqlalchemy import and_, or_, not_

stmt = select(User).where(User.fullname.like("%ha%"))

# несколько условий через and_ / or_
stmt = select(User).where(
    and_(User.fullname.like("%ha%"), User.fullname ≠ 'Shaun Beck')
)

# то же самое: несколько .where() подряд = логическое И
stmt = select(User).where(User.fullname.like("%ha%")).where(User.fullname ≠ 'Shaun Beck')

```

Сортировка

```

from sqlalchemy import desc

stmt = select(User).order_by(User.fullname)           # по возрастанию
stmt = select(User).order_by(desc(User.fullname))    # по убыванию

```

UPDATE и DELETE

```

from sqlalchemy import update, delete

with engine.connect() as connection:
    stmt = update(User).where(User.id == 1).values(age=31)
    connection.execute(stmt)
    connection.commit()

with engine.connect() as connection:
    stmt = delete(User).where(User.id == 1)
    connection.execute(stmt)
    connection.commit()

```

Как и в Core, после операций изменения данных обязательно нужен `connection.commit()` — без него изменения не сохранятся.

Eager и Lazy Loading

Когда у модели есть связи (`relationship`), важно понимать, **когда** SQLAlchemy реально загружает связанные объекты — это сильно влияет на производительность.

Lazy Loading (ленивая загрузка) — поведение по умолчанию

Связанные объекты подгружаются только в момент обращения к ним, отдельным запросом. Плюс — не тратим ресурсы на данные, которые не понадобятся. Минус — проблема **N+1 запросов**: если в цикле обратиться к связи у каждого из N объектов, вместо одного эффективного запроса выполнится N дополнительных.

```

with Session(engine) as session:
    stmt = select(User).where(User.name == "Алиса")
    user = session.scalars(stmt).first() # запрос №1: сам пользователь

    for post in user.posts:              # запрос №2: только тут подгрузятся посты
        print(post.title)

```

Eager Loading (жадная загрузка) — загрузка заранее

Связанные объекты подгружаются сразу вместе с основным — одним (или парой) запросов, без N+1. Полезна, когда точно известно, что связанные данные понадобятся.

Стратегия	Как работает	Когда уместна
<code>joinedload()</code>	Через <code>JOIN</code> , один общий запрос	Связи «один-к-одному», «многие-к-одному»
<code>subqueryload()</code>	Через отдельный подзапрос	Связи «один-ко-многим», «многие-ко-многим»
<code>selectinload()</code>	Отдельный <code>SELECT ... IN (...)</code>	Большое количество связанных объектов

```

from sqlalchemy.orm import joinedload

with Session(engine) as session:
    stmt = select(User).where(User.name == "Алиса").options(joinedload(User.posts))
    user = session.scalars(stmt).first() # один запрос — пользователь + все его посты

    for post in user.posts:
        print(post.title)

```

Что выбрать: Lazy Loading — когда не все связанные данные нужны сразу или важно экономить память. Eager Loading — когда связанные данные точно понадобятся и важно избежать проблемы N+1 / минимизировать число запросов.

Сессии vs execute(): что выбрать

	Сессия (ORM-стиль)	execute() (Core-стиль)
Уровень	Работа с Python-объектами	Ближе к SQL, работа через <code>connection</code>
Удобно для	Сложной логики, отслеживания состояния объектов	Простых запросов, максимального контроля над SQL
Транзакции	Управляются сессией во многом автоматически	Нужно управлять явно (<code>commit()</code> после каждого изменения)

Оба подхода используют одни и те же конструкции SQLAlchemy (`select()` , `where()` , `join()` и т.д.) — отличается только способ выполнения запроса. В одном проекте их можно комбинировать, выбирая то, что удобнее для конкретной задачи.

Расширенные возможности SQLAlchemy

Валидация данных

Валидация — проверка того, что данные соответствуют заданным правилам, прежде чем сохранить их в базу. Это защищает базу от некорректных или неполных значений. В SQLAlchemy для этого можно добавить проверку прямо в модель через декоратор `@validates` .

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, validates

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    username: Mapped[str] = mapped_column(String, nullable=False)
    age: Mapped[int] = mapped_column(Integer, nullable=False)

    @validates("username")
    def validate_username(self, key, username: str) → str:
        if not username or len(username) < 3:
            raise ValueError(f"Field '{key}' must be at least 3 characters long")
        return username

    @validates("age")
    def validate_age(self, key, age: int) → int:
        if age < 18:
            raise ValueError("User must be at least 18 years old")
        return age
```

Метод, помеченный `@validates("поле")` , вызывается автоматически при каждом изменении этого поля. Он получает `self` , `key` (имя поля) и новое значение; должен вернуть валидное значение или выбросить исключение (например, `ValueError`) — тогда сохранение прервётся.

События и колбэки (callback)

Колбэк (callback) — функция, которая передаётся в другую функцию и автоматически вызывается при наступлении определённого события.

SQLAlchemy генерирует события на разных уровнях: **модели** (перед вставкой/обновлением/удалением объекта), **сессии** (например, перед `commit()`) и **engine** (перед выполнением самого SQL-запроса). Чаще всего используются события на уровне модели.

```

from sqlalchemy import event

@event.listens_for(User, "before_insert")
def validate_user(mapper, connection, target):
    if len(target.username) < 3:
        raise ValueError("Username must be at least 3 characters long")
    if target.age < 18:
        raise ValueError("User must be at least 18 years old")

```

Обработчик получает `mapper` (связь модели с таблицей), `connection` и `target` — сам объект, который сохраняется.

Событие	Когда срабатывает
<code>before_insert</code> / <code>after_insert</code>	Перед / после вставки объекта
<code>before_update</code>	Перед обновлением объекта
<code>before_delete</code>	Перед удалением объекта
<code>before_flush</code>	Перед отправкой изменений сессии в базу
<code>after_commit</code>	После успешного завершения транзакции

Важно: события срабатывают только при стандартной работе через ORM (`session.add()` , `session.commit()` , `session.delete()` , `session.flush()`). Прямой `session.execute()` , `query().delete()` , а также массовые операции `bulk_insert_mappings()` / `bulk_update_mappings()` работают в обход ORM и события **не вызывают** — это полезно знать, если валидация через события вдруг «не сработала» при массовой вставке.

Асинхронные запросы (AsyncIO)

С версии 2.0 SQLAlchemy поддерживает `asyncio` (стандартную библиотеку Python для асинхронного программирования) — это особенно полезно в веб-приложениях, где нужно эффективно обрабатывать много запросов одновременно (например, вместе с FastAPI). Для работы нужен асинхронный драйвер базы данных: `asyncpg` (PostgreSQL), `aiomysql` (MySQL) или `aiosqlite` (SQLite).

```
poetry add aiosqlite
```

Асинхронный движок создаётся через `create_async_engine()` , а сессия — через `async_sessionmaker()` :

```

import asyncio
from sqlalchemy import select
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker

async def async_main():
    engine = create_async_engine("sqlite+aiosqlite:/// :memory:")

    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all) # создание таблиц — синхронная операция

    async_session = async_sessionmaker(bind=engine, expire_on_commit=False)

    async with async_session() as session:
        new_user = User(username="Alice", age=25)
        session.add(new_user)
        await session.commit()

        result = await session.execute(select(User))
        for user in result.scalars().all():
            print(user.username, user.age)

asyncio.run(async_main()) # Alice 25

```

Все обращения к базе сопровождаются `await`. Основные правила асинхронной работы:

- Работать только в рамках одного event loop (цикла событий)
- Не вызывать синхронные функции внутри асинхронного кода
- Для «синхронных по своей природе» операций (например, создание/удаление таблиц) использовать `run_sync()`
- Объекты моделей из результата запроса получать через `scalar()` / `scalars()`, как и в синхронном варианте

Работа с транзакциями

Транзакция гарантирует атомарность: все операции внутри неё либо выполняются полностью, либо не выполняются вообще (при ошибке — откат). SQLAlchemy всегда работает в рамках транзакции: она открывается автоматически при начале работы с сессией и закрывается при `session.commit()` (изменения сохраняются) или `session.rollback()` (изменения отменяются).

Удобнее всего использовать транзакцию как контекстный менеджер `session.begin()` — тогда `commit` при успехе и `rollback` при ошибке происходят автоматически:

```

# синхронно
try:
    with session.begin():
        new_user = User(username="Alice", age=25)
        session.add(new_user)
        # commit() вызовется автоматически при выходе из блока
        print("Transaction committed successfully.")
except Exception as e:
    session.rollback()
    print(f"Transaction failed, rolled back. Error: {e}")

```

```
# асинхронно — та же логика, но с await
async with async_session() as session:
    try:
        async with session.begin():
            new_user = User(username="Alice", age=25)
            session.add(new_user)
        print("Transaction committed successfully.")
    except Exception as e:
        await session.rollback()
        print(f"Transaction failed, rolled back. Error: {e}")
```

Миграции схемы базы данных (Alembic)

Миграции — управление изменениями структуры базы данных по мере развития приложения: позволяют применять, откатывать и автоматически генерировать изменения схемы. **Alembic** — инструмент для миграций, который обычно используют в паре с SQLAlchemy (сам SQLAlchemy встроенного механизма миграций не имеет).

Проблема, которую решают миграции: база данных со временем меняется — добавляются поля, таблицы, связи, — а в базе уже могут быть реальные данные, которые нельзя просто потерять. Миграция — это, по сути, скрипт, описывающий переход от одной схемы (A) к другой (B) и обратно.

Главная польза Alembic — он **сам генерирует такие скрипты**, сравнивая текущую схему базы с описанием моделей в коде. Второй важный момент — Alembic хранит полную историю изменений и умеет переключаться на любую точку в этой истории (по аналогии с Git, только не для кода, а для структуры базы).

Установка и настройка

```
pip install alembic
alembic init alembic
```

Команда `alembic init alembic` создаёт папку `alembic` (название — последний аргумент команды) и файл конфигурации `alembic.ini`.

В `alembic.ini` нужно указать строку подключения к базе — параметр `sqlalchemy.url`:

```
sqlalchemy.url = sqlite:///example.db
```

А в файле `alembic/env.py` — импортировать базовый класс моделей `Base`, чтобы Alembic знал, с какой схемой сравнивать базу:

```
from your_project.models import Base

target_metadata = Base.metadata
```

Создание и применение миграций

После изменения моделей (например, добавили новое поле) создаётся новая миграция:

```
alembic revision --autogenerate -m "Description of changes"
```

Флаг `-m` задаёт короткое описание миграции — оно помогает быстро понять суть изменений позже, не читая весь код файла. Флаг `--autogenerate` заставляет Alembic сравнить текущую схему базы с `Base.metadata` и сгенерировать код изменений автоматически. Файл миграции появляется в папке `alembic/versions`.

Автосгенерированную миграцию обязательно нужно открыть и проверить глазами: Alembic не всегда корректно улавливает все изменения — например, изменения в связях (`relationship`) между таблицами он может пропустить. При необходимости код миграции правится вручную.

Применение и откат миграций:

Команда	Что делает
<code>alembic upgrade head</code>	Применяет все ещё не применённые миграции, база доводится до последней версии
<code>alembic downgrade -1</code>	Откатывает на одну миграцию назад
<code>alembic downgrade <revision_id></code>	Откатывает до конкретной версии (id ревизии видно в файлах внутри <code>alembic/versions</code>)

Практический пример: TODO-проект с Alembic

Соберём небольшой проект-планировщик задач: таблица `notes` (заметки/задачи), у каждой заметки — пункты в таблице `records` (один-ко-многим), а также теги в таблице `tags` (многие-ко-многим через связку `note_m2m_tag`).

Подготовка проекта

```
mkdir example_alembic
cd example_alembic

poetry init
poetry add sqlalchemy alembic
poetry shell

alembic init migrations
```

`alembic init migrations` создаёт папку `migrations` (с подпапкой `versions` для файлов миграций) и файл `alembic.ini` в корне проекта.

Модели

Файл `models.py` — три модели и таблица-связка для отношения многие-ко-многим:

```

from datetime import datetime
from sqlalchemy import ForeignKey, Table, Column, Integer, PrimaryKeyConstraint, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

# связь many-to-many между notes и tags
note_m2m_tag = Table(
    "note_m2m_tag", Base.metadata,
    Column("note_id", Integer, ForeignKey("notes.id", ondelete="CASCADE")),
    Column("tag_id", Integer, ForeignKey("tags.id", ondelete="CASCADE")),
    PrimaryKeyConstraint("note_id", "tag_id"),
)

class Note(Base):
    __tablename__ = "notes"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(nullable=False)
    created: Mapped[datetime] = mapped_column(default=func.now())
    records: Mapped[list["Record"]] = relationship(cascade="all, delete", back_populates="note")
    tags: Mapped[list["Tag"]] = relationship(secondary=note_m2m_tag, back_populates="notes")

class Record(Base):
    __tablename__ = "records"
    id: Mapped[int] = mapped_column(primary_key=True)
    description: Mapped[str] = mapped_column(nullable=False)
    done: Mapped[bool] = mapped_column(default=False)
    note_id: Mapped[int] = mapped_column(ForeignKey("notes.id", ondelete="CASCADE"))
    note: Mapped["Note"] = relationship(back_populates="records")

class Tag(Base):
    __tablename__ = "tags"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(nullable=False, unique=True)
    notes: Mapped[list["Note"]] = relationship(secondary=note_m2m_tag, back_populates="tags")

```

Важное отличие от прошлых примеров: здесь нет вызова `Base.metadata.create_all(engine)` — теперь за создание таблиц полностью отвечает Alembic, через миграции.

Подключение к базе — файл `connect.py` :

```

from sqlalchemy.engine import create_engine
from sqlalchemy.orm import sessionmaker

url_to_db = "sqlite:///mynotes.db"
engine = create_engine(url_to_db)
Session = sessionmaker(bind=engine)
session = Session()

```

Настройка env.py

В файле `alembic.ini` есть строка `sqlalchemy.url` — в неё можно прописать путь к базе напрямую, но удобнее и безопаснее задавать URL программно в `migrations/env.py` (особенно если в будущем URL будет храниться в переменных окружения, а не в открытом виде в файле):

```

from logging.config import fileConfig
from sqlalchemy import engine_from_config, pool
from alembic import context

from models import Base
from connect import url_to_db

target_metadata = Base.metadata
config.set_main_option("sqlalchemy.url", url_to_db)

```

`target_metadata` нужен, чтобы работала автогенерация миграций — Alembic сравнивает базу именно с этими метаданными. `config.set_main_option()` переопределяет URL из `alembic.ini` программно.

`env.py` поддерживает два режима: **online** (по умолчанию) — миграции применяются сразу через подключение к базе; **offline** — включается флагом `--sql` (например, `alembic upgrade head --sql > migration.sql`) и просто генерирует SQL-файл без подключения к базе — удобно, если миграцию нужно передать и выполнить вручную на другой машине.

Первая миграция

```
alembic revision --autogenerate -m 'Init'
```

Alembic сравнит модели с (пока пустой) базой и сгенерирует файл миграции в `migrations/versions`. Внутри — функции `upgrade()` и `downgrade()` с командами создания/удаления таблиц:

```

def upgrade() → None:
    op.create_table('notes',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('name', sa.String(), nullable=False),
        sa.Column('created', sa.DateTime(), nullable=False),
        sa.PrimaryKeyConstraint('id')
    )
    # ...аналогично создаются tags, note_m2m_tag, records

def downgrade() → None:
    op.drop_table('records')
    op.drop_table('note_m2m_tag')
    op.drop_table('tags')
    op.drop_table('notes')

```

Применяем миграцию — создаётся файл базы `mynotes.db` и в нём все таблицы:

```
alembic upgrade head
```

При первой миграции Alembic сам создаёт служебную таблицу `alembic_version` с одним столбцом — `id` последней применённой миграции. По ней Alembic понимает, какие миграции уже применены, а какие ещё нет. Менять эту таблицу вручную нельзя.

Наполнение данными и запрос

Файл `seeds.py` — добавляем заметку с пунктами и тегами:

```

from connect import session
from models import Note, Record, Tag

tag1, tag2 = Tag(name="groceries"), Tag(name="food")
note = Note(name="Go to the store")
note.tags = [tag1, tag2]

Record(description="Buy bread", note=note)
Record(description="Buy sausage 0.5 kg", note=note)

session.add(note)
session.commit()

```

Запрос с джойном через связку многие-ко-многим (`main.py`):

```

from sqlalchemy import select
from connect import session
from models import Note, Tag, Record, note_m2m_tag

result = (
    session.execute(
        select(Note.id, Note.name, Record.description, Record.done, Tag.name.label("tag"))
        .join(Record).join(note_m2m_tag).join(Tag)
        .filter(Tag.name == "food")
    )
    .mappings().all()  # .mappings() — результат сразу в виде словарей
)

```

```
[{'id': 1, 'name': 'Go to the store', 'description': 'Buy bread', 'done': False, 'tag': 'food'}, ...]
```

Изменение схемы: вторая миграция

Тот же цикл повторяется при каждом изменении моделей: меняем поле в `models.py` → генерируем миграцию → применяем:

```

alembic revision --autogenerate -m "Add description to notes"
alembic upgrade head

# если что-то пошло не так — откат на шаг назад,
# данные, накопленные в базе, при этом никуда не пропадают
alembic downgrade -1

```

Итог: Alembic отслеживает каждое изменение схемы и умеет откатывать его назад; умеет генерировать миграции автоматически по изменениям моделей; при этом ничто не мешает при необходимости дописать сложную миграцию вручную.